SOFTWARE DEVELOPMENT WITH STANDARDS

SPRINT 1 – AUTHENTICATION & RBAC DESIGN

AYSU AKDÜZEN

2200004962

1. Introduction and Summary

Secure authentication and authorization are essential for protecting sensitive information in modern systems. Many applications rely on centralized authentication services instead of storing user credentials locally.

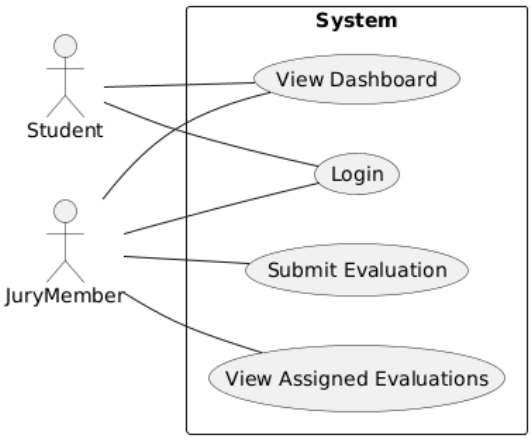
In this project, for a prototype, web application was developed that authenticates users through the university's CATS portal. After successful authentication, the system assigns roles to users and controls access based on these roles. Jury members are only allowed to access evaluations assigned to them.

2. System Overview

The system is a mobile-based evaluation system. Users access the system through a web browser, while the server processes authentication and authorization logic.

The application communicates with the university's CATS authentication system to verify user credentials. After authentication, the system assigns a role to the user and controls access to different resources based on that role.

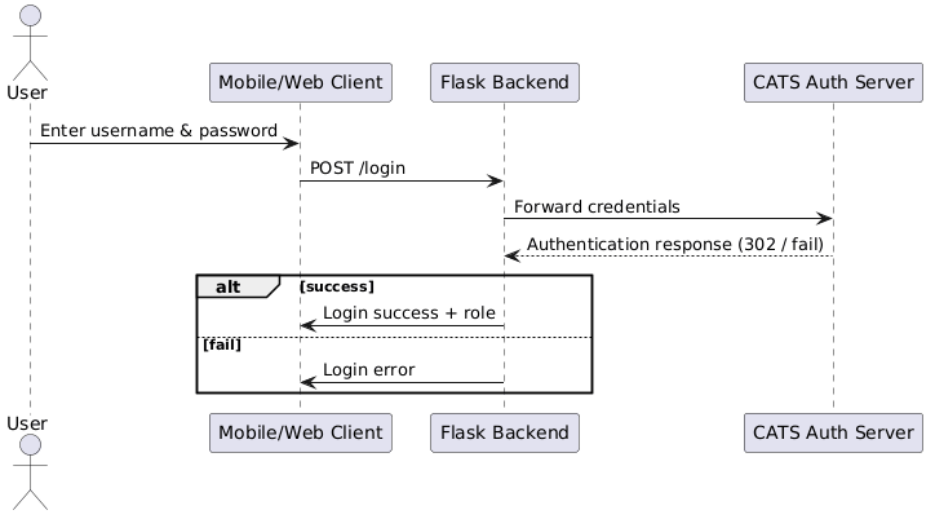
The prototype implementation focuses on backend security design by Flask while the final system will include a dedicated mobile interface developed with Flutter. (The security-critical logic is intentionally implemented on the server side so that the same backend can securely support different client platforms such as web and mobile.)



3. Authentication Design

Authentication is handled through the university's CATS portal. When a user enters their credentials, the application sends an HTTP POST request to the CATS login endpoint.

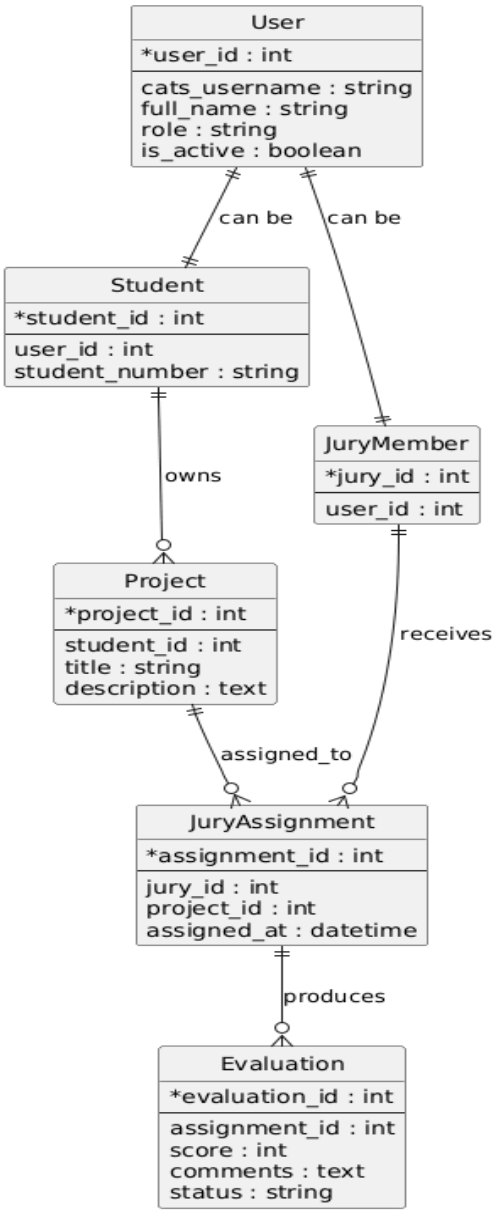
If the response indicates a successful login, the user is considered authenticated. This method improves security because user passwords are not stored in the application.



4. System Data Model

This section presents the conceptual data model of the proposed system. The model defines the main entities involved in the evaluation process, including users, students, jury members, projects, assignments, and evaluations. It illustrates how evaluation responsibilities are assigned to specific jury members through structured relationships.

The data model supports the implementation of role-based authorization by ensuring that each evaluation is linked to a particular jury assignment and project ownership. This structure helps maintain data integrity, access control, and clear separation of responsibilities within the system.



5. Role-Based Authorization

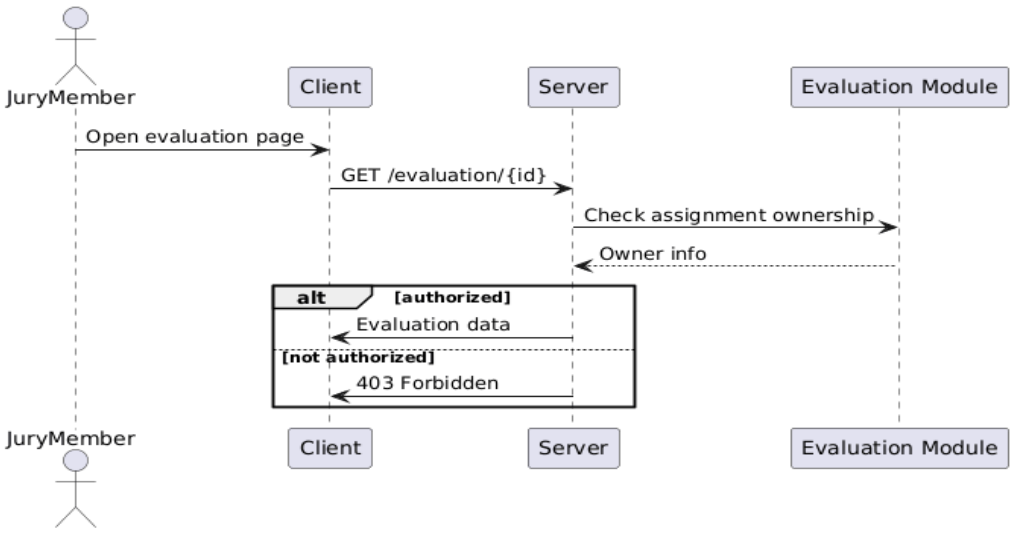
After authentication, the system assigns a role to the user. The main roles in the system are Student and JuryMember.

Access to system resources depends on the user's role. Students have limited access, while jury members can access evaluation pages. Unauthorized access attempts result in a “403 Forbidden” response.

In addition to the Student and Jury roles, an Admin role is defined. Admin accounts are authenticated locally, independent of CATS, so that administrative operations remain available even when the CATS service is unreachable. The resulting role–permission matrix is given in Table 1.

Table 1 – Role–permission matrix

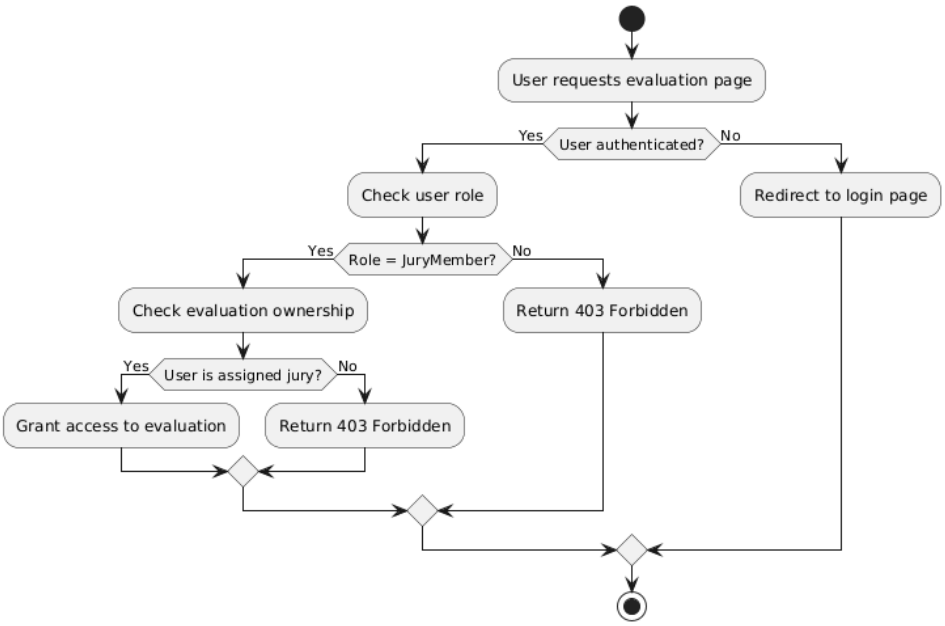
Operation	Admin	Jury	Student
Login method	Local login	CATS	CATS
View projects	All projects	Assigned projects only	Own project only
Enter / submit evaluation scores	No	Own evaluations only	No
Access another jury member's evaluation	Read only	No (403 Forbidden)	No (403 Forbidden)
Manage users, projects and jury assignments	Yes	No	No
View final result	All results	–	Own result (after publication)



6. Evaluation Access Control

Each evaluation is assigned to a specific jury member. The system checks whether the logged-in user is the assigned owner of the evaluation.

If the user is not authorized to access the evaluation, the system blocks the request and returns a “403 Forbidden” message.



This figure illustrates the evaluation access control flow of the system. When a user attempts to access an evaluation, the system first verifies authentication status and user role. If the user is a jury member, the system checks whether the requested evaluation is assigned to that specific jury member. Access is granted only when all authorization conditions are satisfied; otherwise, the system returns a forbidden response or redirects the user to the login page.

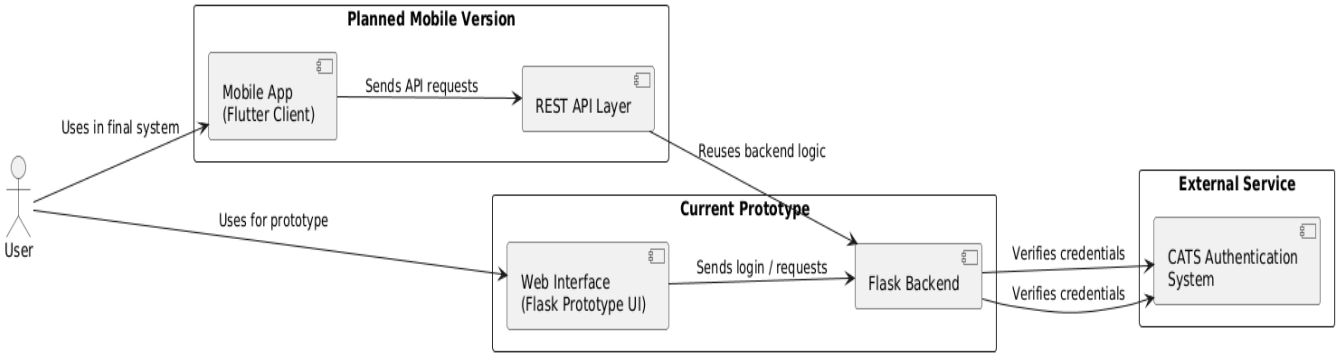
7. Implementation

In this phase of the project, a simple web-based prototype was developed to validate the authentication and authorization mechanisms of the proposed system. The main purpose of this implementation was not to provide the final user interface, but to demonstrate that the security logic of the system functions correctly.

The prototype was implemented using Python and the Flask web framework. User credentials entered through a basic web interface are forwarded to the university's CATS authentication service. If the response indicates a successful login, the system assigns an internal role to the authenticated user and applies role-based authorization rules.

In addition, resource-level access control is enforced by verifying whether a jury member is authorized to access a specific evaluation. This ensures that evaluation data can only be viewed or submitted by the assigned jury member.

Although the current implementation uses a web interface for demonstration purposes, the overall architecture is designed to support a mobile-based system. In the planned mobile version, the same backend logic can be reused by exposing RESTful API endpoints, while a Flutter-based mobile client will provide the user interface. Therefore, this prototype serves as a proof-of-concept for secure backend functionality within the system.



8. Related Database Design

The users, projects, project_members and jury_assignments tables used by the authorization rules are defined in the database design document (docs/database/schema_design.pdf). Ownership checks are performed by matching jury_id against the authenticated user identifier.